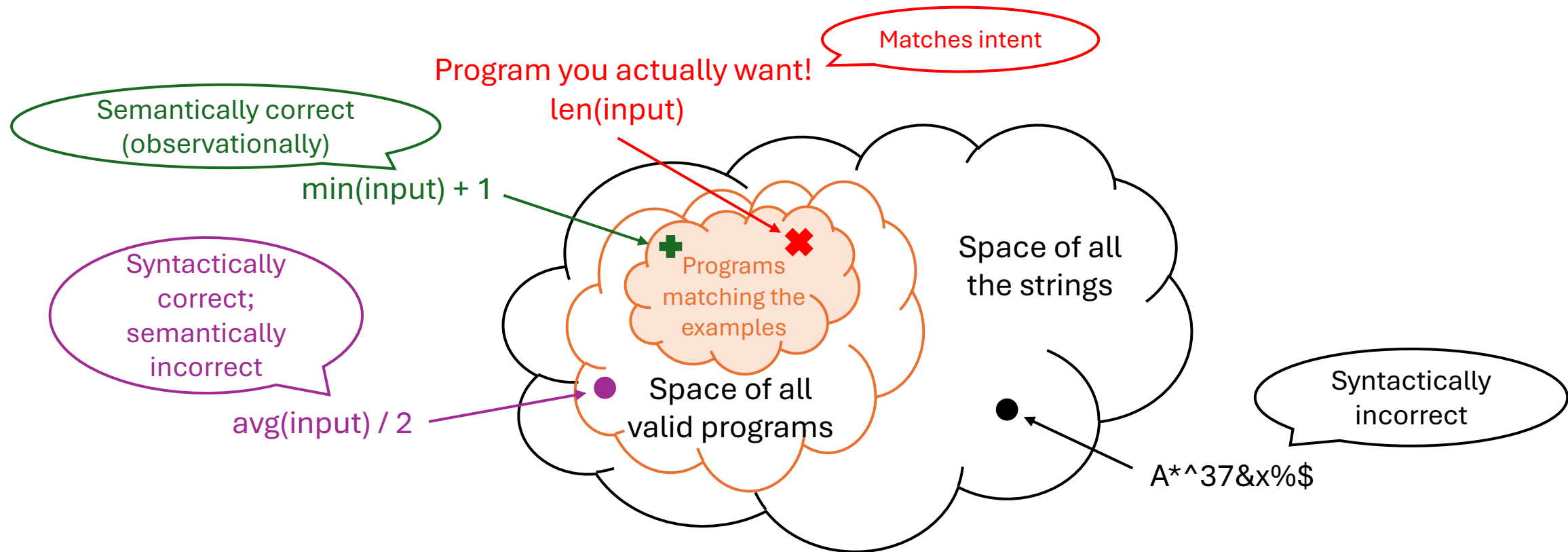


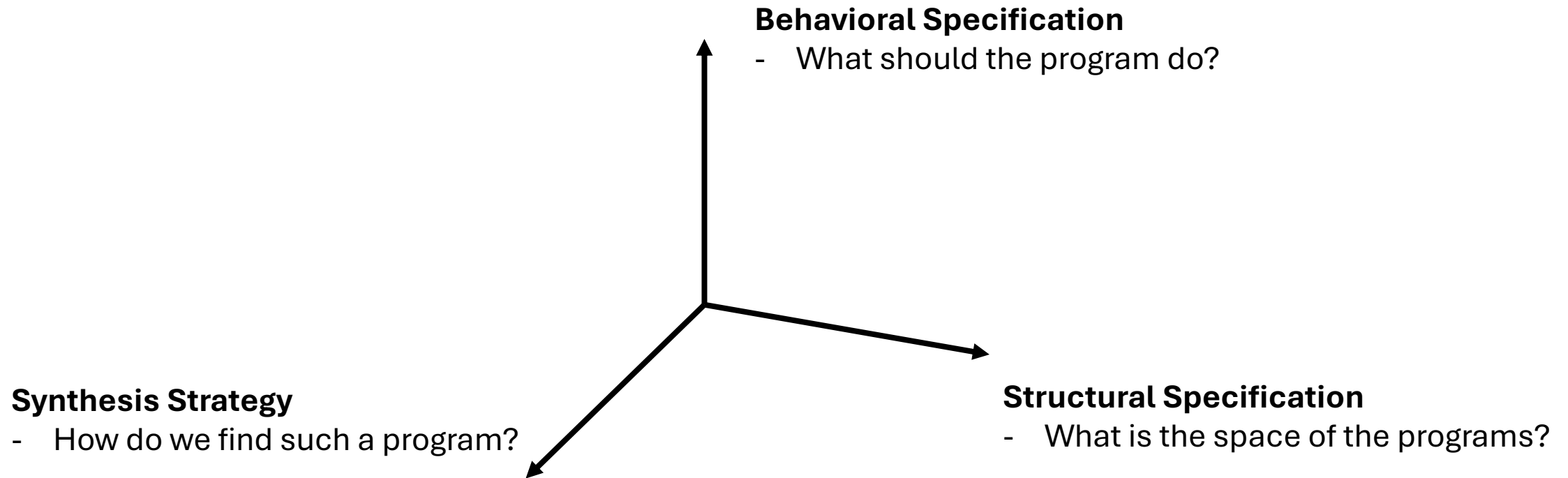
Machine Programming

Lecture 3 – Bottom-up and Top-Down Enumerative Synthesis

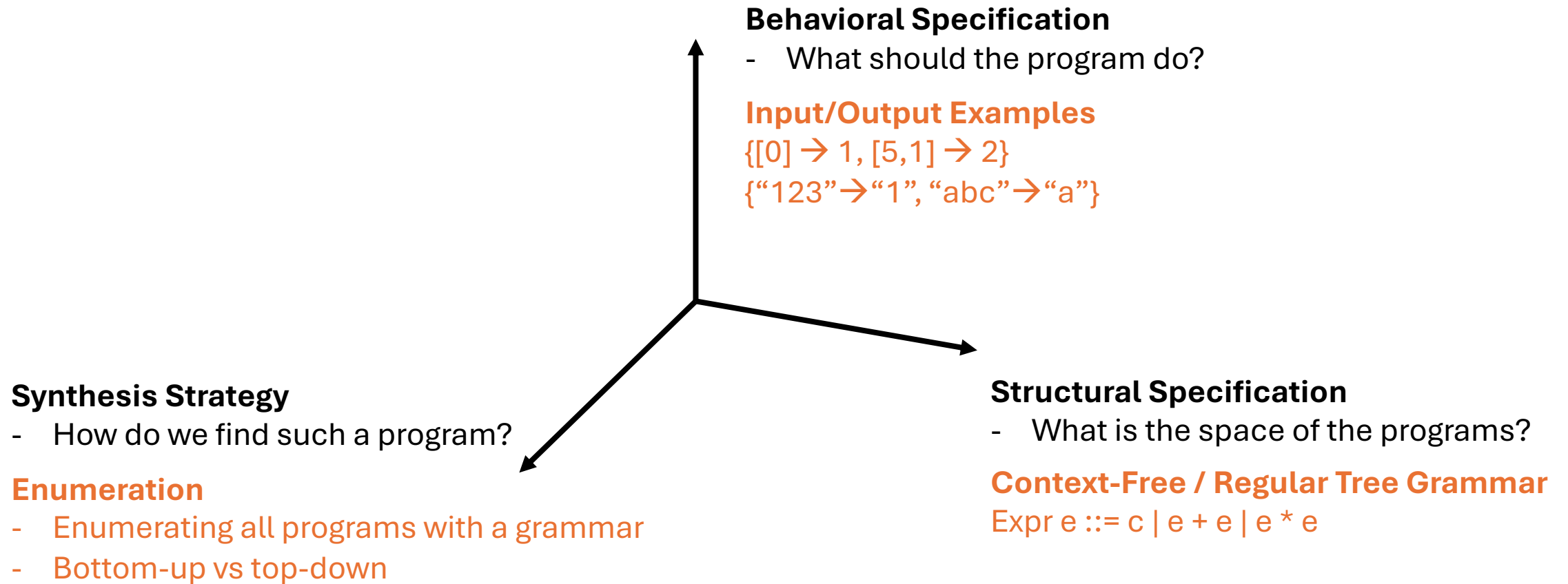
Ziyang Li



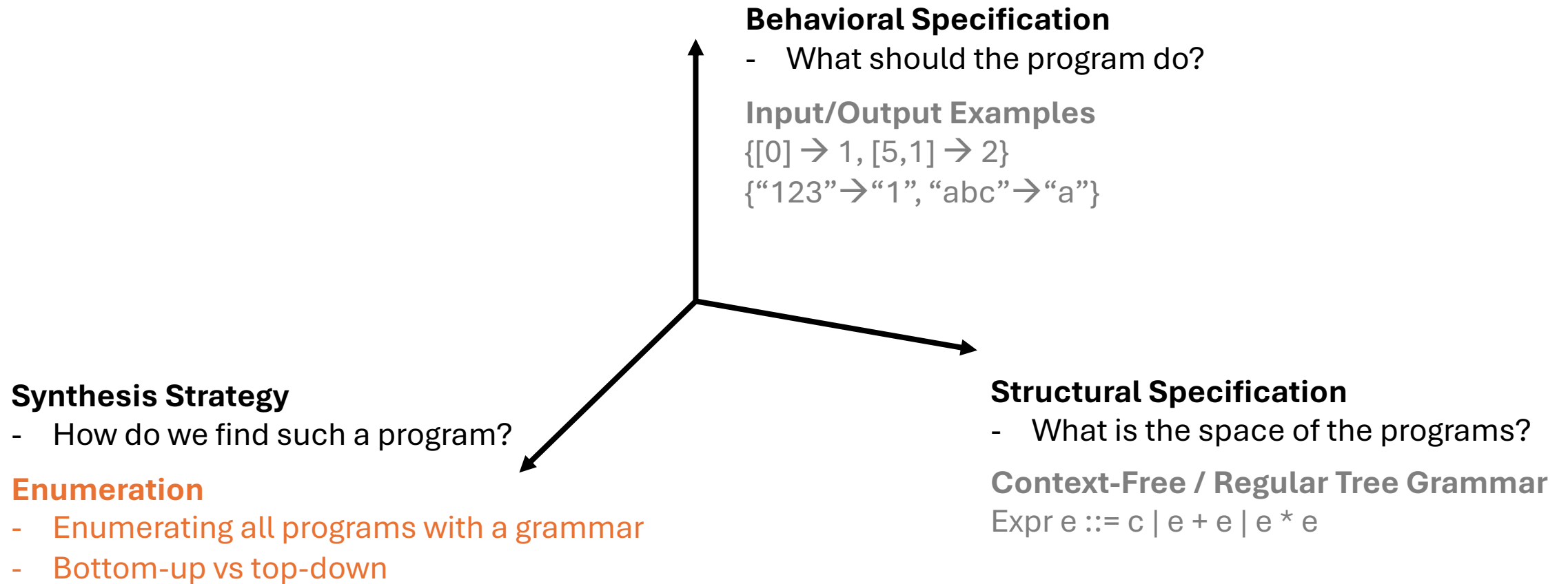
Dimensions in Program Synthesis



Today



Today



Enumerative search

- **Idea**: enumerate programs from the grammar one by one and test them on the examples
- **Challenge**: How do we systematically enumerate all programs?
 - Bottom-up
 - Top-down

Bottom-up enumeration

- Maintain a **bank** of complete programs
 - Starting from all the terminal symbols
- Combine programs in the bank using **production rules**
 - Applying all possible production rules at each iteration

```
(Program)  $\underline{P} ::= L \mid N$   
  (List)  $\underline{L} ::= \text{input}$   
            $\mid \text{empty}$   
            $\mid \text{single}(\underline{N})$   
            $\mid \text{concat}(\underline{L}, \underline{L})$   
  (Number)  $\underline{N} ::= \text{len}(\underline{L})$   
               $\mid \text{min}(\underline{L})$   
               $\mid \text{add}(\underline{N}, \underline{N})$   
               $\mid 0 \mid 1 \mid 2 \mid \dots$ 
```

Bottom-up enumeration: algorithm

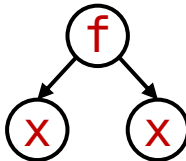
```
bottom-up(< $\Sigma$ , N, R, S>, [ $i \rightarrow o$ ], max_depth):  
    bank := {}  
    for depth in [0..max_depth]:  
        forall rule in R:  
            forall new_prog in grow(rule, depth, bank):  
                if ( $A = S \wedge$  new_prog( $[i]$ ) =  $[o]$ ):  
                    return new_program  
            insert new_program to bank;  
  
grow( $A \rightarrow \sigma(A_1 \dots A_k)$ , d, bank):  
    if ( $d = 0 \wedge k = 0$ ) yield  $\sigma$  // terminal  
    else forall  $\langle t_1, \dots, t_k \rangle$  in bankk: // cartesian product  
        if  $A_i \rightarrow^* t_i$ :  
            yield  $\sigma(t_1, \dots, t_k)$ 
```

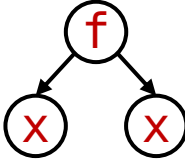
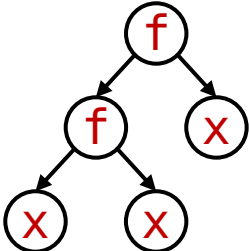
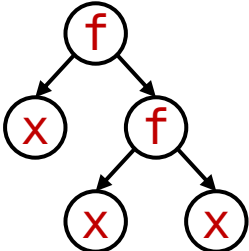
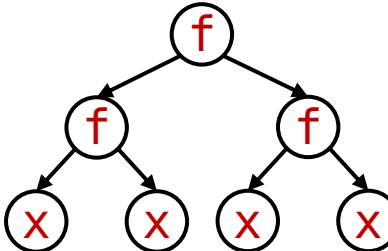
```
(Program)  $\underline{P} ::= L \mid N$   
      (List)  $L ::=$  input  
                | empty  
                | single( $N$ )  
                | concat( $L, L$ )  
      (Number)  $N ::=$  len( $L$ )  
                | min( $L$ )  
                | add( $N, N$ )  
                | 0 | 1 | 2 | ...
```

Bottom-up enumeration: example

$E ::= x \mid f(E, E)$

Depth ≤ 0 

Depth ≤ 1  

Depth ≤ 2     

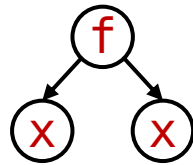
Bottom-up enumeration: example

$E ::= x \mid f(E, E)$

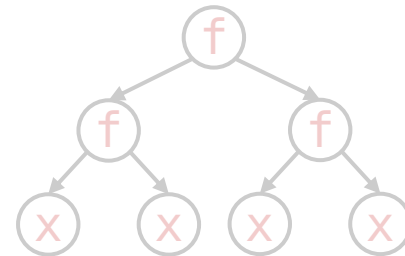
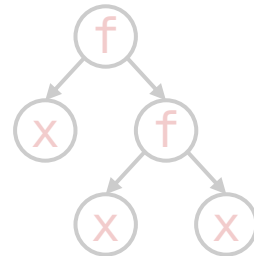
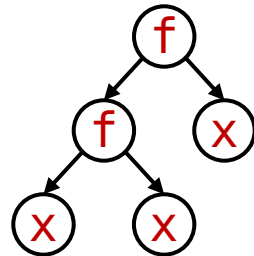
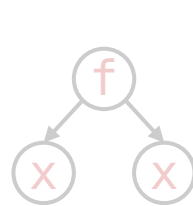
Depth ≤ 0



Depth ≤ 1

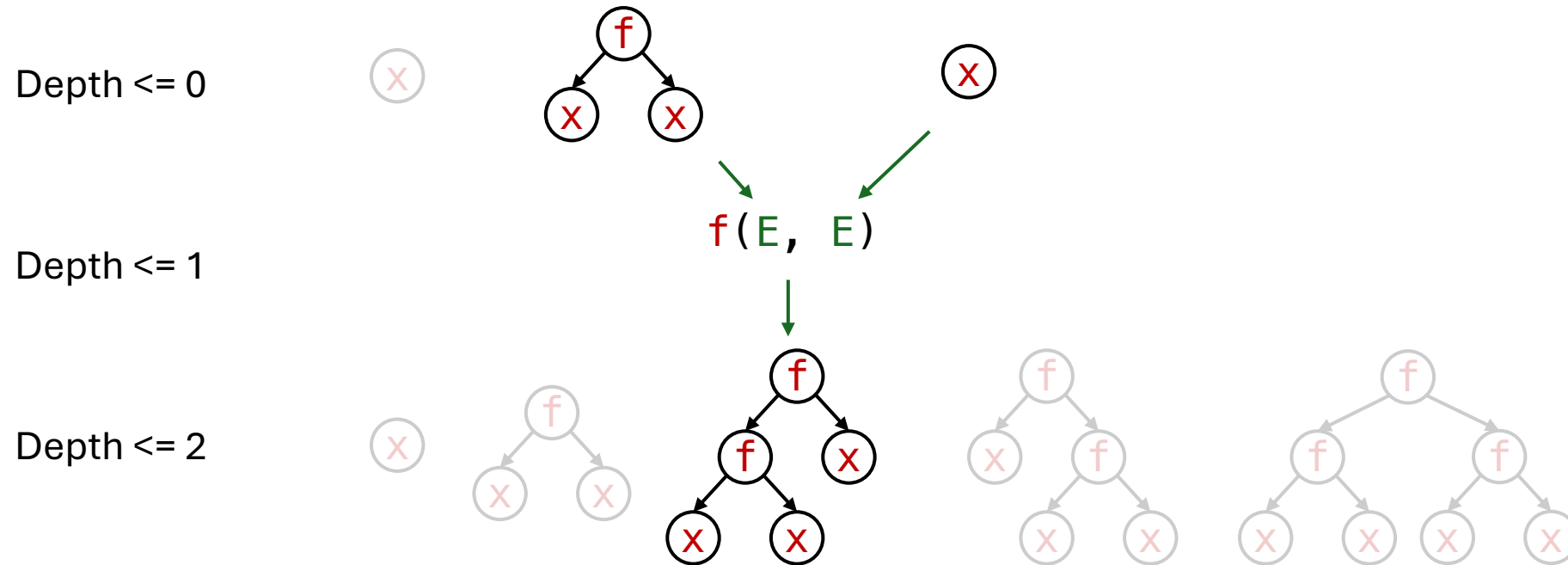


Depth ≤ 2



Bottom-up enumeration: example

$E ::= x \mid f(E, E)$

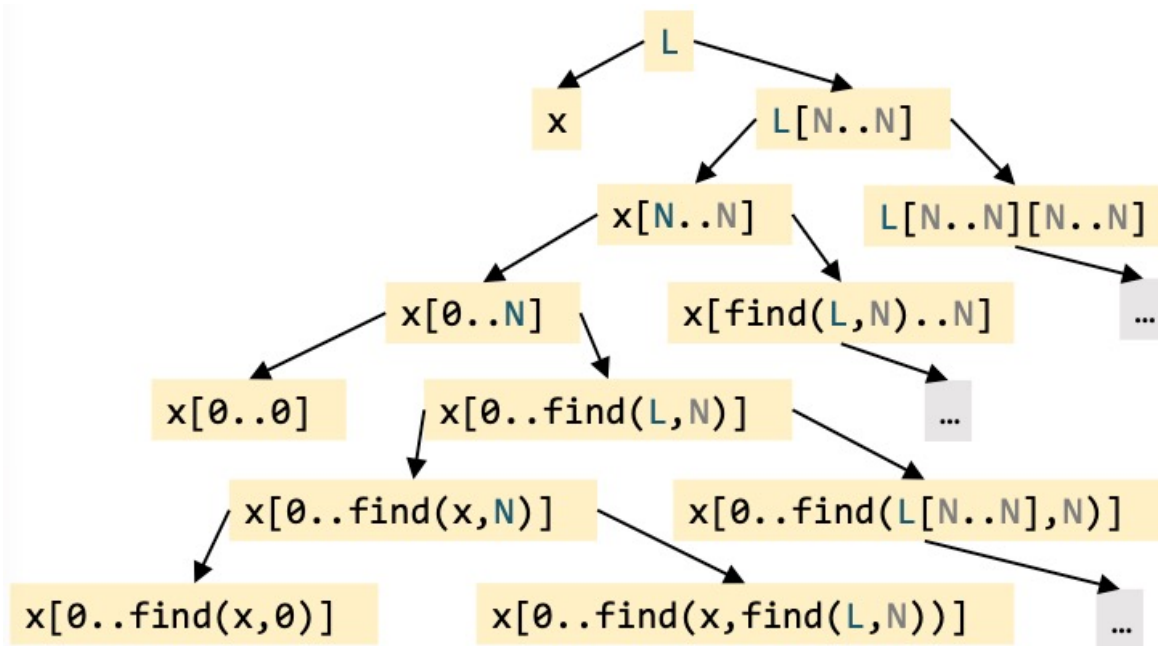


Top-down enumeration

- Search space is a tree, where:
 - Nodes are whole incomplete programs
 - Edges are derivations in a single step

```

(List)  L ::= L[N:N]
        | x // input
(Number) N ::= find(L, N)
        | 0
    
```

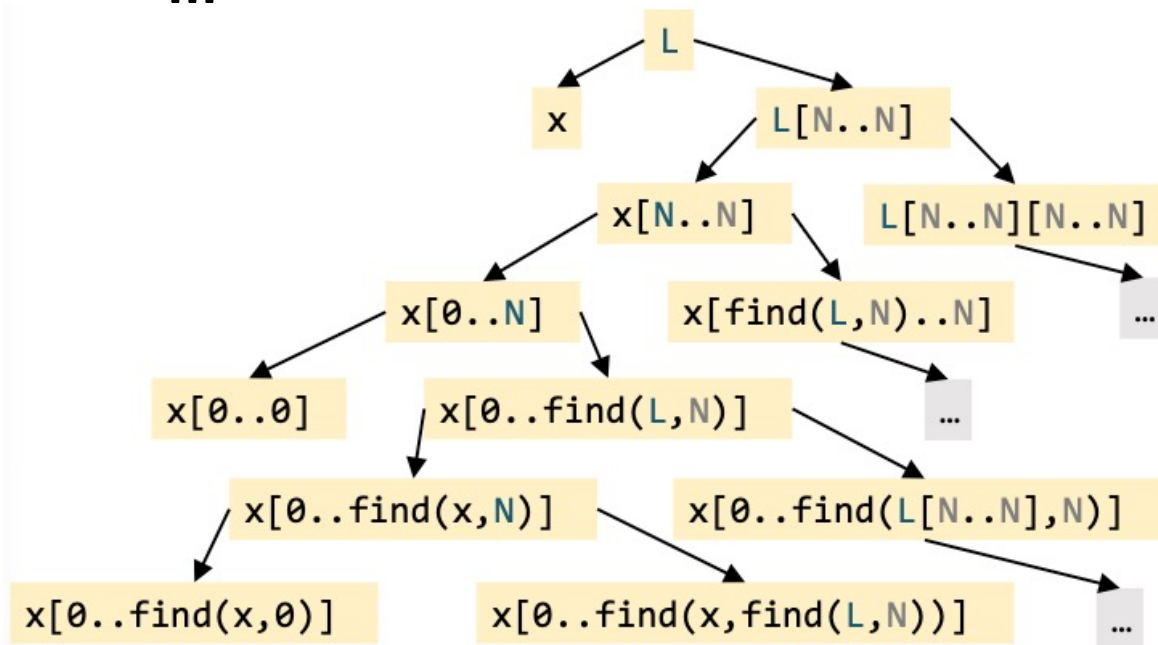


Top-down enumeration

- Search tree can be traversed
 - Depth-first
 - Breadth-first
 - ...

```

(List)  L ::= L[N:N]
        | x // input
(Number) N ::= find(L, N)
        | 0
  
```



Bottom-up vs top-down

Top-down

Program candidates have roots on **starting non-terminal symbol** but might not be **complete**

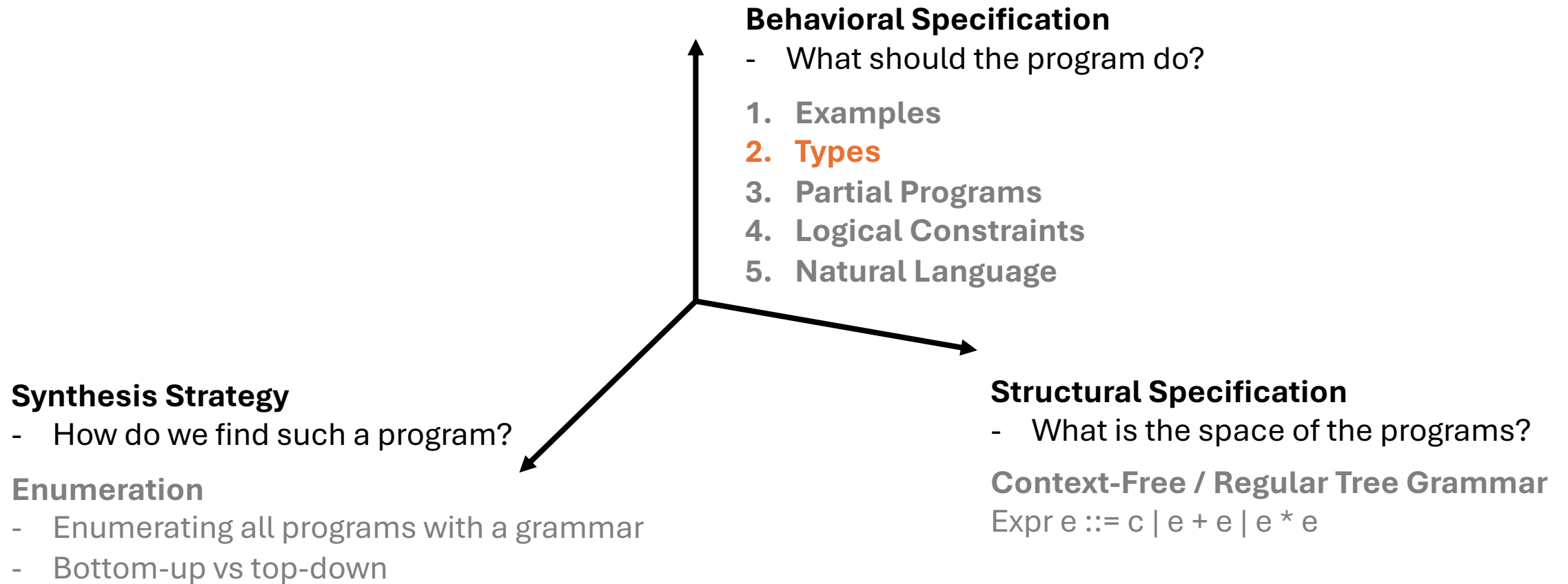
- Cannot always run on inputs
- Can always relate to outputs

Bottom-up

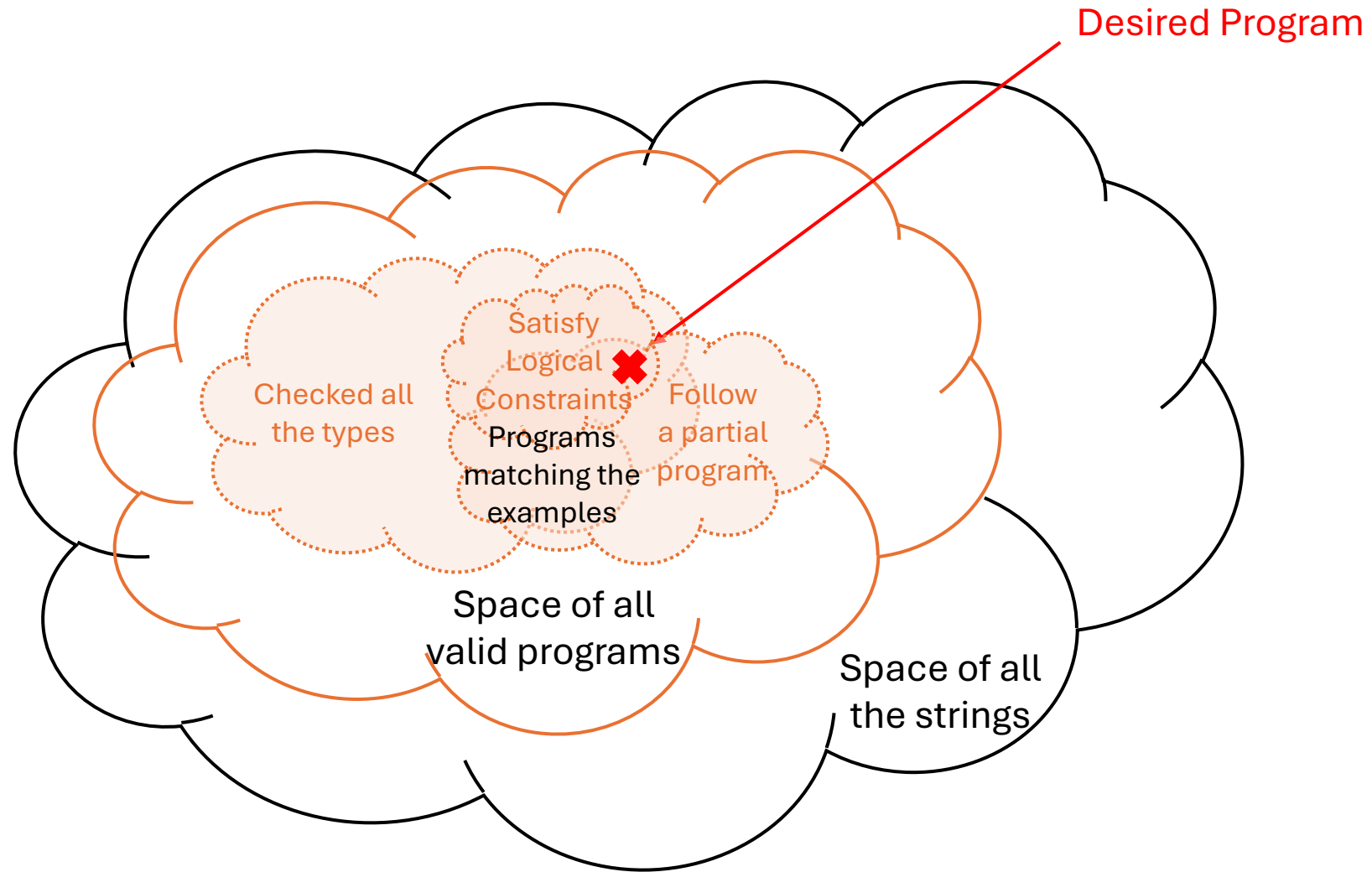
Program candidates are **complete** but might not be **whole**

- Can always run on inputs
- Cannot always relate to outputs

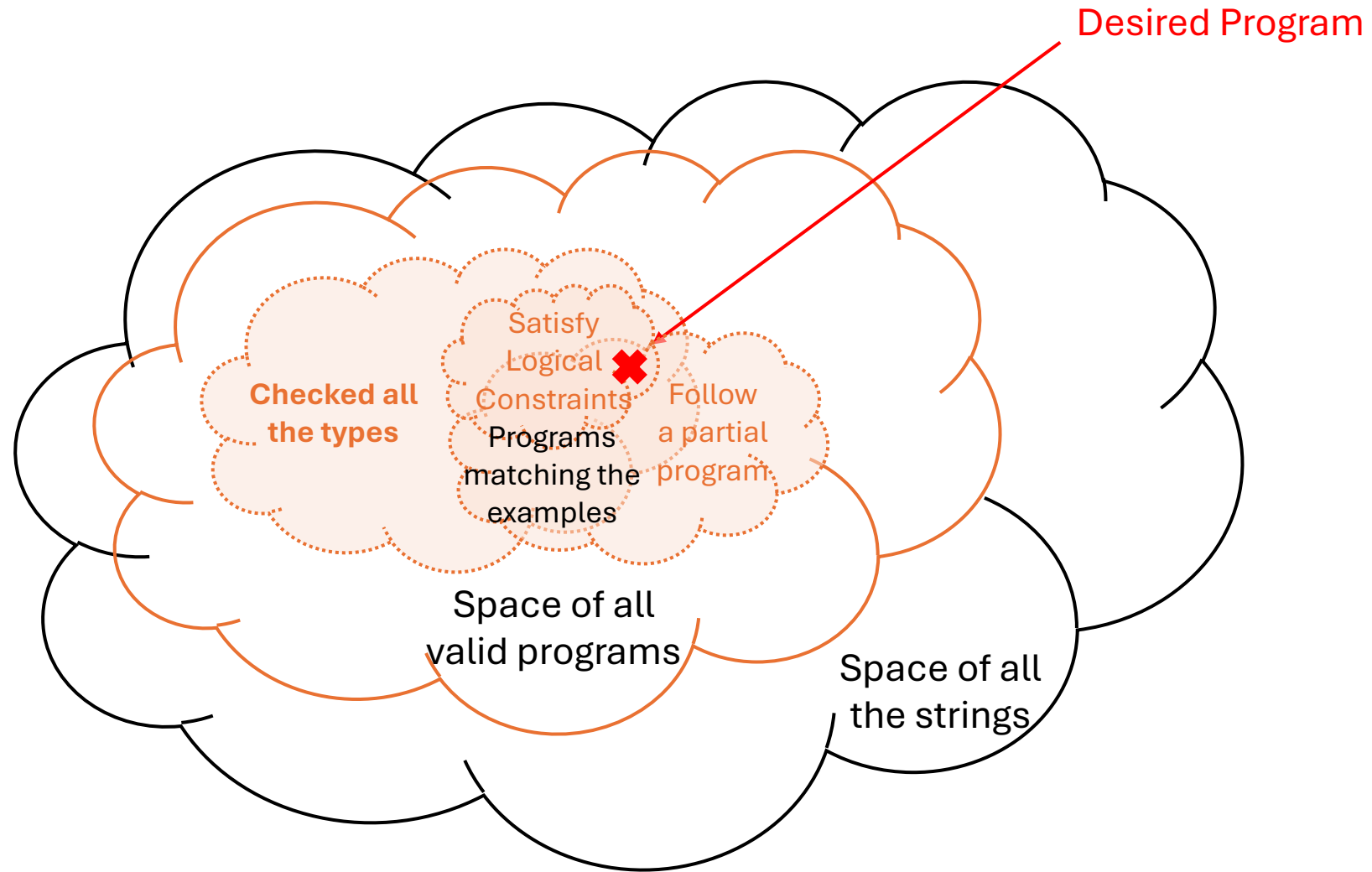
Today



High Level Picture



High Level Picture



Specification: Types

- Fundamentals
 - Example: A Domain Specific Language (DSL) on List Manipulation
 - Type System
 - Type Checking
 - Pruning Top-Down Search with Types
- Advanced
 - Polymorphic Types
 - Refinement Types

List Manipulation DSL

Synthesis Goal:

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

List Manipulation DSL

Synthesis Goal:

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

`dropmins`: for each inner list corresponding to a list of grades, drop the lowest grade

List Manipulation DSL

`dropmins`: for each inner list corresponding to a list of grades, drop the lowest grade

Input	Output
<pre>[[71, 75, 83], [90, 87, 95], [68, 77, 80]]</pre>	<pre>[[75, 83], [90, 95], [77, 80]]</pre>

```
def dropmins(input: List[List[int]]) -> List[List[int]]:  
  # implementation
```

List Manipulation DSL

dropmins: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = []  
        min_num = min(grades)  
        for grade in grades:  
            if grade > min_num:  
                dropmin.append(grade)  
        outputs.append(dropmin)  
    return outputs
```

List Manipulation DSL

dropmins: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = []  
        min_num = min(grades)  
        for grade in grades:  
            if grade > min_num:  
                dropmin.append(grade)  
        outputs.append(dropmin)  
    return outputs
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = [g for g in grades if g > min(grades)]  
  
        outputs.append(dropmin)  
    return outputs
```

List Manipulation DSL

dropmins: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = []  
        min_num = min(grades)  
        for grade in grades:  
            if grade > min_num:  
                dropmin.append(grade)  
        outputs.append(dropmin)  
    return outputs
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = [  
            g for g in grades  
            if g > min(grades)  
        ]  
        outputs.append(dropmin)  
    return outputs
```

List Manipulation DSL

dropmins: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = []  
        min_num = min(grades)  
        for grade in grades:  
            if grade > min_num:  
                dropmin.append(grade)  
        outputs.append(dropmin)  
    return outputs
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = [  
            g for g in grades  
            if g > min(grades)  
        ]  
        outputs.append(dropmin)  
    return outputs
```

```
def dropmins(input: ...) -> ...:  
    outputs = [  
        [g for g in grades  
         if g > min(grades)]  
        for grades in input  
    ]  
    return outputs
```

List Manipulation DSL

dropmins: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = []  
        min_num = min(grades)  
        for grade in grades:  
            if grade > min_num:  
                dropmin.append(grade)  
        outputs.append(dropmin)  
    return outputs
```

```
def dropmins(input: ...) -> ...:  
    outputs = []  
    for grades in input:  
        dropmin = [  
            g for g in grades  
            if g > min(grades)  
        ]  
        outputs.append(dropmin)  
    return outputs
```

```
def dropmins(input: ...) -> ...:  
    return [  
        g for g in grades  
        if g > min(grades)  
    ]  
    for grades in input
```

List Manipulation DSL

`dropmins`: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
  return [  
    [  
      g for g in grades  
      if g > min(grades)  
    ]  
    for grades in input  
  ]
```

```
def dropmins(input: ...) -> ...:  
  return [  
    filter(grades, lambda g: g > min(grades))  
    for grades in input  
  ]
```

List Manipulation DSL

`dropmins`: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
  return [  
    filter(  
      grades,  
      lambda g: g > min(grades))  
    for grades in input  
  ]
```

```
def dropmins(input: ...) -> ...:  
  return map(input, lambda grades:  
    filter(grades, lambda g:  
      g > min(grades))  
  )
```

List Manipulation DSL

`dropmins`: for each inner list corresponding to a list of grades, drop the lowest grade

Input

```
[ [71, 75, 83],  
  [90, 87, 95],  
  [68, 77, 80] ]
```

Output

```
[ [75, 83],  
  [90, 95],  
  [77, 80] ]
```

```
def dropmins(input: ...) -> ...:  
  return map(input, lambda grades:  
    filter(grades, lambda g:  
      g > min(grades)  
    )  
  )
```

```
def dropmins(input: ...) -> ...:  
  return map(input, lambda grades:  
    filter(grades, lambda g:  
      g > reduce(grades, 0, lambda acc, cur:  
        acc if acc < cur else cur  
      )  
    )  
  )
```

```
def dropmins(input: ...) -> ...:
    return map(input, lambda grades:
        filter(grades, lambda g:
            g > reduce(grades, 0, lambda acc, cur:
                acc if acc < cur else cur
            )
        )
    )
```

map

filter

reduce

```
def dropmins(input: ...) -> ...:
    return map(input, lambda grades:
        filter(grades, lambda g:
            g > reduce(grades, 0, lambda acc, cur:
                acc if acc < cur else cur
            )
        )
    )
```

map : [U] -> (U -> V) -> [V]

filter : [U] -> (U -> bool) -> [U]

reduce : [U] -> V -> (V -> U -> V) -> V

map : $[U] \rightarrow (U \rightarrow V) \rightarrow [V]$

Take a list of U, apply the mapping function $U \rightarrow V$, obtaining a list of V.

filter : $[U] \rightarrow (U \rightarrow \text{bool}) \rightarrow [U]$

Take a list of U, use the filtering function $U \rightarrow \text{bool}$ to keep only the Us that produces true.
The result is again a list of U.

reduce : $[U] \rightarrow V \rightarrow (V \rightarrow U \rightarrow V) \rightarrow V$

Take a list of U and an initial state V. From the left, continuously apply the transition function $V \rightarrow U \rightarrow V$ to accumulate the state V. After traversing the entire list, return the final V.

Typing Practices

sum

min

len

sort

Typing Practices

`sum` : [Int] -> Int

`min` : [Int] -> Int

`len` : [Int] -> Int

`sort` : [Int] -> [Int]

Implementation Practices

`sum` : `[Int] -> Int`

`min` : `[Int] -> Int`

`len` : `[Int] -> Int`

`sort` : `[Int] -> [Int]`

Your Tools:

`map` : `[U] -> (U -> V) -> [V]`

`filter` : `[U] -> (U -> bool) -> [U]`

`reduce` : `[U] -> V -> (V -> U -> V) -> V`

Implementation Practices

```
sum          : [Int] -> Int
    sum(list) = reduce(list, 0, lambda acc, n: acc + n)
min          : [Int] -> Int

len          : [Int] -> Int

sort         : [Int] -> [Int]
```

Your Tools:

```
map          : [U] -> (U -> V) -> [V]
filter       : [U] -> (U -> bool) -> [U]
reduce       : [U] -> V -> (V -> U -> V) -> V
```

Implementation Practices

`sum` : `[Int] -> Int`

`sum(list) = reduce(list, 0, lambda acc, n: acc + n)`

`min` : `[Int] -> Int`

`min(list) = reduce(list, None, lambda acc, n: n if acc is None or n < acc else acc)`

`len` : `[Int] -> Int`

`sort` : `[Int] -> [Int]`

Your Tools:

`map` : `[U] -> (U -> V) -> [V]`

`filter` : `[U] -> (U -> bool) -> [U]`

`reduce` : `[U] -> V -> (V -> U -> V) -> V`

Implementation Practices

`sum` : `[Int] -> Int`

`sum(list) = reduce(list, 0, lambda acc, n: acc + n)`

`min` : `[Int] -> Int`

`min(list) = reduce(list, None, lambda acc, n: n if acc is None or n < acc else n)`

`len` : `[Int] -> Int`

`len(list) = reduce(list, 0, lambda acc, n: acc + 1)`

`sort` : `[Int] -> [Int]`

Your Tools:

`map` : `[U] -> (U -> V) -> [V]`

`filter` : `[U] -> (U -> bool) -> [U]`

`reduce` : `[U] -> V -> (V -> U -> V) -> V`

Implementation Practices

`sum` : `[Int] -> Int`

`sum(list) = reduce(list, 0, lambda acc, n: acc + n)`

`min` : `[Int] -> Int`

`min(list) = reduce(list, None, lambda acc, n: n if acc is None or n < acc else n)`

`len` : `[Int] -> Int`

`len(list) = reduce(list, 0, lambda acc, n: acc + 1)`

`sort` : `[Int] -> [Int]`

???

Your Tools:

`map` : `[U] -> (U -> V) -> [V]`

`filter` : `[U] -> (U -> bool) -> [U]`

`reduce` : `[U] -> V -> (V -> U -> V) -> V`

```

dropmins(input) =
  map(input, lambda grades:
    filter(grades, lambda g:
      g > reduce(grades, 0, lambda acc, cur:
        acc if acc < cur else cur
      )
    )
  )

map      : [U] -> (U -> V) -> [V]
filter   : [U] -> (U -> bool) -> [U]
reduce   : [U] -> V -> (V -> U -> V) -> V

```

List Manipulation DSL Syntax

```

Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...

Var ::= String

BinaryOp ::= > | < | == | + | - | ...

```

Typing Checking

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...

Var ::= String

BinaryOp ::= > | < | == | + | - | ...
```

```
true + map(3, list)
```

```
1 + (lambda x: x + 1)
```

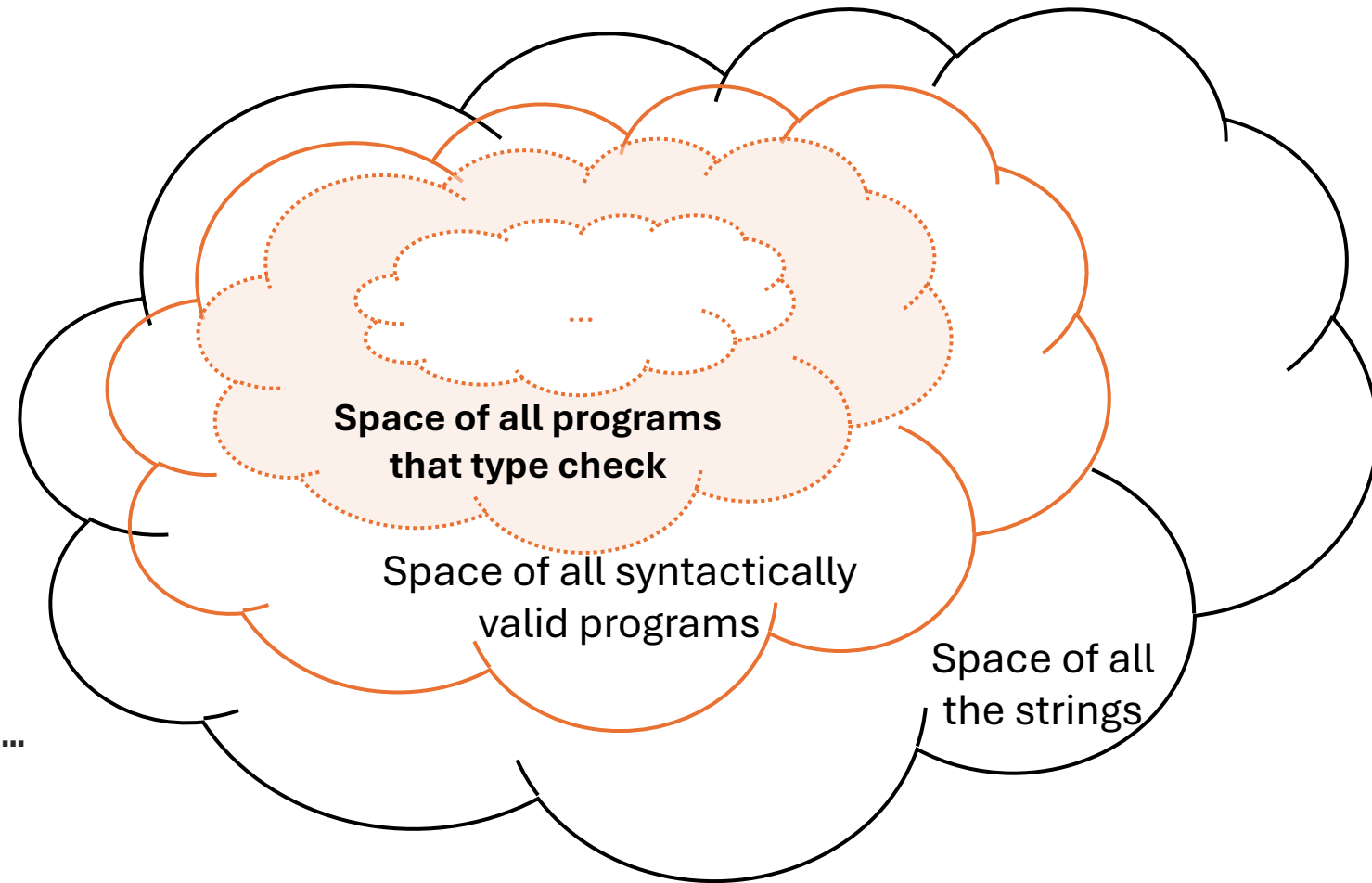
```
map(input, lambda x: x + 1)
```

Typing Checking

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...

Var ::= String

BinaryOp ::= > | < | == | + | - | ...
```



Types

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

```
Var ::= String
```

```
BinaryOp ::= > | < | == | + | - | ...
```

```
Type ::= Int
      | Bool
      | List[Type]
      | Type -> Type
```

Typing Rule

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

```
Type ::= Int
      | Bool
      | List[Type]
      | Type -> Type
```

$$\frac{\langle \text{Premises} \rangle}{\langle \text{Context} \rangle \vdash \langle \text{Expr} \rangle : \langle \text{Type} \rangle}$$

Typing Rule

$$\frac{\langle \text{Premises} \rangle}{\langle \text{Context} \rangle \vdash \langle \text{Expr} \rangle : \langle \text{Type} \rangle}$$

```
Expr ::= Var
      | lambda Var. Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

```
Type ::= Int
      | Bool
      | List[Type]
      | Type -> Type
```

$$[\text{Int}] \frac{}{C \vdash 1 : \text{Int}}$$

$$[\text{True}] \frac{}{C \vdash \text{true} : \text{Bool}}$$

$$[\text{False}] \frac{}{C \vdash \text{false} : \text{Bool}}$$

Typing Rule

```
Expr ::= Var
      | lambda Var. Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

$$\text{[Arith]} \frac{C \vdash e_1:\text{Int} \quad C \vdash e_2:\text{Int}}{C \vdash e_1 + e_2 : \text{Int}}$$
$$\frac{\langle \text{Premises} \rangle}{\langle \text{Context} \rangle \vdash \langle \text{Expr} \rangle : \langle \text{Type} \rangle}$$

```
Type ::= Int
      | Bool
      | List[Type]
      | Type -> Type
```

$$\text{[Equality]} \frac{C \vdash e_1:\text{Int} \quad C \vdash e_2:\text{Int}}{C \vdash e_1 == e_2 : \text{Bool}}$$

Typing Rule

```
Expr ::= Var
      | lambda Var. Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

$$[\text{Var}] \frac{(v \mapsto \tau) \in C}{C \vdash \text{Var}(v) : \tau}$$
$$\frac{\langle \text{Premises} \rangle}{\langle \text{Context} \rangle \vdash \langle \text{Expr} \rangle : \langle \text{Type} \rangle}$$

```
Type ::= Int
      | Bool
      | List[Type]
      | Type -> Type
```

$$[\text{Lambda}] \frac{C \cup \{x \mapsto \tau_1\} \vdash e : \tau_2}{C \vdash \lambda x. e : \tau_1 \rightarrow \tau_2}$$

Typing Rule

```
Expr ::= Var
      | lambda Var. Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

$$[\text{Map}] \frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \tau_2}{C \vdash \text{map}(e_1, e_2) : \text{List}[\tau_2]}$$
$$\frac{\langle \text{Premises} \rangle}{\langle \text{Context} \rangle \vdash \langle \text{Expr} \rangle : \langle \text{Type} \rangle}$$

```
Type ::= Int
      | Bool
      | List[Type]
      | Type -> Type
```

$$[\text{Filter}] \frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \text{Bool}}{C \vdash \text{filter}(e_1, e_2) : \text{List}[\tau_1]}$$

Type Checking that Succeeds

```
len : [Int] -> Int
```

```
  len(list) = reduce(list, 0, lambda acc, n: acc + 1)
```

Type Checking that Succeeds

```
len : [Int] -> Int
```

```
len(list) = reduce(list, 0, lambda acc, n: acc + 1)
```

$$\begin{array}{c}
\frac{\dots}{C \vdash \text{len} : [\text{Int}] \rightarrow \text{Int}} \quad \checkmark \\
\hline
\begin{array}{c}
\text{...} \\
\frac{C \vdash \text{reduce}(\text{list}, 0, \text{lambda acc, n. acc} + 1): \text{Int}}{C \vdash \text{len} : [\text{Int}] \rightarrow \text{Int}} \\
\hline
\begin{array}{c}
C \vdash \text{list} : \text{List}[\text{Int}] \quad C \vdash 0 : \text{Int} \quad C \vdash \text{lambda acc, n. acc} + 1 : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \\
\hline
\frac{\begin{array}{c}
\frac{(\text{list} \mapsto \text{List}[\text{Int}]) \in C}{C \vdash \text{list} : \text{List}[\text{Int}]} \quad \frac{(\text{acc} \mapsto \text{Int}) \in C}{C \vdash 0 : \text{Int}} \quad \frac{(\text{acc} \mapsto \text{Int}) \in C \quad (\text{n} \mapsto \text{Int}) \in C}{C \vdash \text{lambda acc, n. acc} + 1 : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}}
\end{array}}{C \vdash \text{len} : [\text{Int}] \rightarrow \text{Int}}
\end{array}
\end{array}$$

Type Checking that Failed

```
gt0 : [Int] -> [Int]
gt0(list) = filter(list, 0)
```

$$[\text{Filter}] \frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \text{Bool}}{C \vdash \text{filter}(e_1, e_2) : \text{List}[\tau_1]}$$

Type Checking that Failed

```
gt0 : [Int] -> [Int]
gt0(list) = filter(list, 0)
```

$$\text{[Filter]} \frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \text{Bool}}{C \vdash \text{filter}(e_1, e_2) : \text{List}[\tau_1]}$$

$$\frac{\frac{(list \mapsto \text{List}[\text{Int}]) \in C}{C \vdash list : \text{List}[\text{Int}]} \quad \frac{}{C \vdash 0 : \text{Int}}}{C \vdash \text{filter}(list, 0) : \text{X}}$$

Type Checking that Succeeded (Corrected)

```
gt0 : [Int] -> [Int]
gt0(list) = filter(list, lambda x: x > 0)
```

$$\text{[Filter]} \frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \text{Bool}}{C \vdash \text{filter}(e_1, e_2) : \text{List}[\tau_1]}$$

$$\frac{\frac{(list \mapsto \text{List}[\text{Int}]) \in C}{C \vdash list : \text{List}[\text{Int}]}}{\quad} \quad \frac{\frac{(x \mapsto \text{Int}) \in C}{C \vdash x : \text{Int}} \quad \frac{}{C \vdash 0 : \text{Int}} \quad \frac{(x \mapsto \text{Int}) \in C}{C \vdash x : \text{Int}}}{C \vdash x > 0 : \text{Bool}} \quad \frac{}{C \vdash \text{lambda } x. x > 0 : \text{Int} \rightarrow \text{Bool}}$$

$$\frac{\dots \quad C \vdash \text{filter}(list, \text{lambda } x. x > 0) : \text{List}[\text{Int}]}$$

$$C \vdash \text{gt0} : \text{List}[\text{Int}] \rightarrow \text{List}[\text{Int}] \quad \checkmark$$

Typing Rules

```

Expr ::= Var
       | lambda Var. Expr
       | filter(Expr, Expr)
       | map(Expr, Expr)
       | reduce(Expr, Expr, Expr)
       | Expr if Expr else Expr
       | Expr BinaryOp Expr
       | true | false
       | 0 | 1 | 2 | ...

Type ::= Int
       | Bool
       | List[Type]
       | Type -> Type
    
```

[Int] $\frac{}{C \vdash \text{IntLiteral} : \text{Int}}$

[Arith] $\frac{C \vdash e_1 : \text{Int} \quad C \vdash e_2 : \text{Int}}{C \vdash e_1 + e_2 : \text{Bool}}$

[Var] $\frac{(v \mapsto \tau) \in C}{C \vdash \text{Var}(v) : \tau}$

[Map] $\frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \tau_2}{C \vdash \text{map}(e_1, e_2) : \text{List}[\tau_2]}$

[True] $\frac{}{C \vdash \text{true} : \text{Bool}}$

[False] $\frac{}{C \vdash \text{false} : \text{Bool}}$

[Equality] $\frac{C \vdash e_1 : \text{Int} \quad C \vdash e_2 : \text{Int}}{C \vdash e_1 == e_2 : \text{Bool}}$

[Lambda] $\frac{C \cup \{x \mapsto \tau_1\} \vdash e : \tau_2}{C \vdash \lambda x. e : \tau_1 \rightarrow \tau_2}$

[Filter] $\frac{C \vdash e_1 : \text{List}[\tau_1] \quad C \vdash e_2 : \tau_1 \rightarrow \text{Bool}}{C \vdash \text{filter}(e_1, e_2) : \text{List}[\tau_1]}$

Typing Rules in the Wild

Datafun [Arntzenius et. al. 2016]

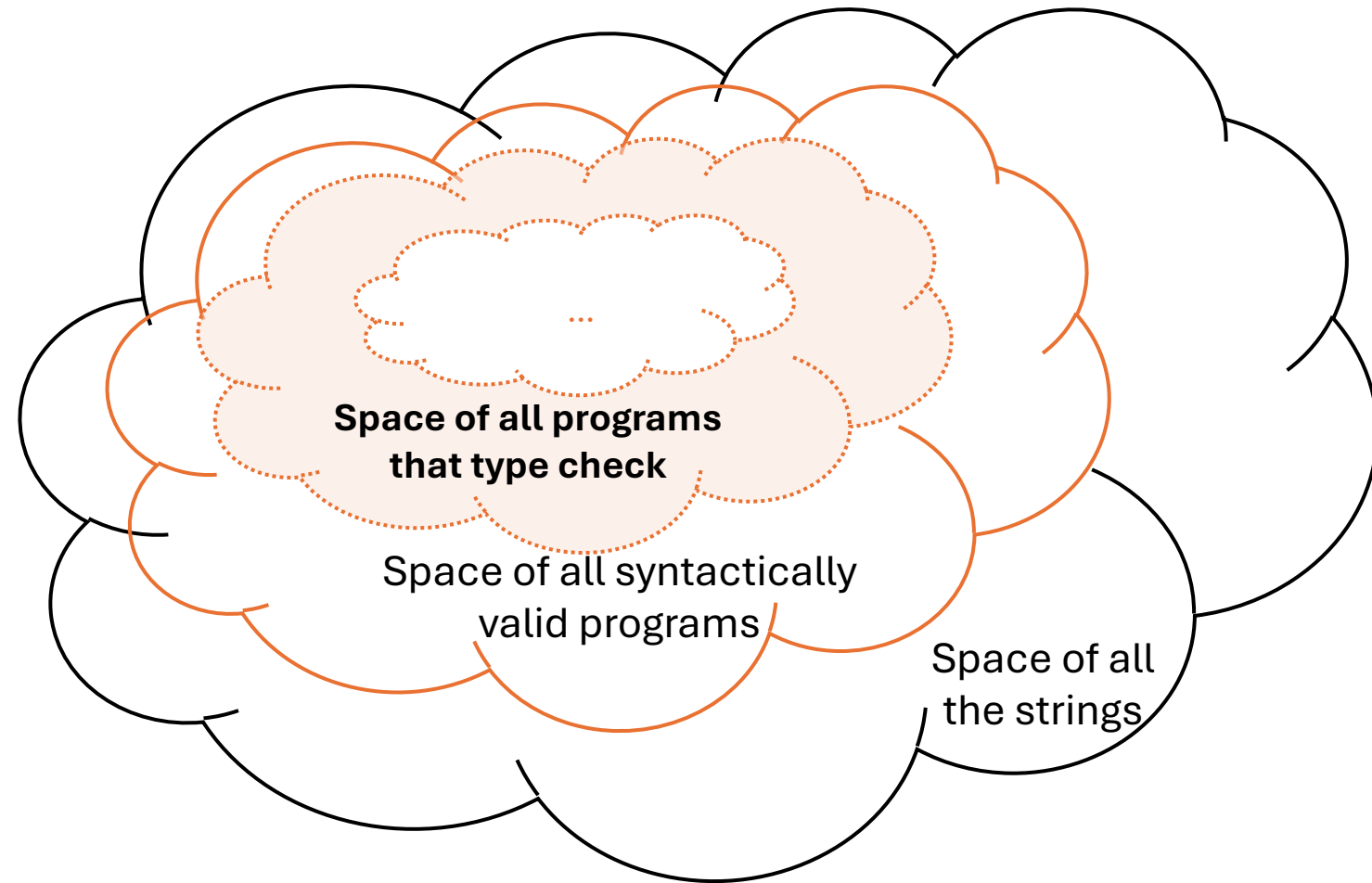
WebAssembly [Haas et. al. 2017]

$\frac{\Delta; \Gamma \vdash \mathcal{L} \Rightarrow \Delta' \quad \Delta, \Delta'; \Gamma \vdash e : L}{\Delta; \Gamma \vdash \bigvee(\mathcal{L}) e : L}$ $\frac{\Delta; \Gamma \vdash \mathcal{L} \Rightarrow \Delta' \quad \Delta, \Delta'; \Gamma \vdash e : A}{\Delta; \Gamma \vdash \{e \mid \mathcal{L}\} : \{A\}}$ $\frac{\Delta; \cdot \vdash e : A \quad \Delta; \Gamma \vdash p : A \Rightarrow \Delta' \quad \Delta, \Delta'; \Gamma \vdash e_1 : C \quad \Delta; \Gamma}{\Delta; \Gamma \vdash \text{case } e \text{ of } p \rightarrow e_1; _ \rightarrow e_2 : C}$	(contexts) $C ::= \{\text{func } tf^*, \text{global } tg^*, \text{table } n^?, \text{memory } n^?, \text{local } t^*, \text{label } (t^*)^*, \text{return } (t^*)^?\}$ Typing Instructions $\frac{}{C \vdash t.\text{const } c : \epsilon \rightarrow t} \quad \frac{}{C \vdash t.\text{unop} : t \rightarrow t} \quad \frac{}{C \vdash t.\text{binop} : t t \rightarrow t} \quad \frac{}{C \vdash t.\text{testop} : t \rightarrow \text{i32}} \quad \frac{}{C \vdash t.\text{relop} : t t \rightarrow \text{i32}}$ $\frac{t_1 \neq t_2 \quad sx^? = \epsilon \Leftrightarrow (t_1 = \text{in} \wedge t_2 = \text{in}' \wedge t_1 < t_2) \vee (t_1 = \text{fn} \wedge t_2 = \text{fn}')}{C \vdash t_1.\text{convert } t_2.sx^? : t_2 \rightarrow t_1} \quad \frac{t_1 \neq t_2 \quad t_1 = t_2 }{C \vdash t_1.\text{reinterpret } t_2 : t_2 \rightarrow t_1}$
$\frac{\Delta; \Gamma \vdash \mathcal{L}_1 \Rightarrow \Delta_1 \quad \Delta; \Gamma \vdash \mathcal{L}_2 \Rightarrow \Delta_2}{\Delta; \Gamma \vdash \mathcal{L}_1, \mathcal{L}_2 \Rightarrow \Delta_1, \Delta_2}$ $\frac{\Delta; \Gamma \vdash e : \{A\} \quad \Delta; \Gamma \vdash p : A \Rightarrow \Delta}{\Delta; \Gamma \vdash p \in e \Rightarrow \Delta'}$ $\frac{\Delta; \Gamma \vdash _ : A}{\Delta; \Gamma \vdash _ : A}$ $\frac{\Delta; \Gamma \vdash p_1 : A_1 \Rightarrow \Delta_1 \quad \Delta; \Gamma \vdash (p_1, p_2) : A_1 \times \Delta}{\Delta; \Gamma \vdash !e : A_{eq} \Rightarrow \cdot}$	$\Sigma; \Delta; \Gamma; \Theta \vdash e : \tau \Rightarrow \Gamma'$ T-MOVE $\Delta; \Gamma; \Theta \vdash_{\text{uniq}} \pi \Rightarrow \{ \text{uniq } \pi \}$ $\Gamma(\pi) = \tau^{\text{SI}} \quad \text{noncopyable}_{\Sigma} \tau^{\text{SI}}$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{n} : \text{u32} \Rightarrow \Gamma$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{\pi} : \tau^{\text{SI}} \Rightarrow \Gamma[\pi \mapsto \tau^{\text{SI}}]$ T-COPY $\Delta; \Gamma; \Theta \vdash_{\text{shrd}} p \Rightarrow \{ \bar{\ell} \}$ $\Delta; \Gamma \vdash_{\text{shrd}} p : \tau^{\text{SI}} \quad \text{copyable}_{\Sigma} \tau^{\text{SI}}$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{p} : \tau^{\text{SI}} \Rightarrow \Gamma$ T-BORROW $\Gamma(r) = \emptyset \quad \Gamma; \Theta \vdash r \text{ rnic}$ $\Delta; \Gamma; \Theta \vdash_{\omega} p \Rightarrow \{ \bar{\ell} \} \quad \Delta; \Gamma \vdash_{\omega} p : \tau^{\text{SI}}$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{\&r \omega p} : \&r \omega \tau^{\text{SI}} \Rightarrow \Gamma[r \mapsto \{ \bar{\ell} \}]$ T-LETREGION $\Sigma; \Delta; \Gamma, r \mapsto \{ \}; \Theta \vdash \boxed{e} : \tau^{\text{SI}} \Rightarrow \Gamma', r \mapsto \{ \bar{\ell} \}$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{\text{letrgn } \langle r \rangle \{ e \}} : \tau^{\text{SI}} \Rightarrow \Gamma'$ T-BRANCH $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{e_1} : \text{bool} \Rightarrow \Gamma_1 \quad \Sigma; \Delta; \Gamma_1; \Theta \vdash \boxed{e_2} : \tau_2^{\text{SI}} \Rightarrow \Gamma_2$ $\Sigma; \Delta; \Gamma_1; \Theta \vdash \boxed{e_3} : \tau_3^{\text{SI}} \Rightarrow \Gamma_3 \quad \tau_2^{\text{SI}} \vee \tau_3^{\text{SI}} = \tau_3^{\text{SI}}$ $\Delta; \Gamma_2; \Theta \vdash^+ \tau_2^{\text{SI}} \leadsto \tau_2^{\text{SI}} \vdash \Gamma_2' \quad \Delta; \Gamma_3; \Theta \vdash^+ \tau_3^{\text{SI}} \leadsto \tau_3^{\text{SI}} \vdash \Gamma_3' \quad \Gamma_2' \cup \Gamma_3' = \Gamma'$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{\text{if } e_1 \{ e_2 \} \text{ else } \{ e_3 \}} : \tau^{\text{SI}} \Rightarrow \Gamma'$ T-SEQ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{e_1} : \tau_1^{\text{SI}} \Rightarrow \Gamma_1$ $\Sigma; \Delta; \Gamma; \text{gc-loans}_{\Theta}(\Gamma_1); \Theta \vdash \boxed{e_2} : \tau_2^{\text{SI}} \Rightarrow \Gamma_2$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{e_1; e_2} : \tau_2^{\text{SI}} \Rightarrow \Gamma_2$
T-LET $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{e_1} : \tau_1^{\text{SI}} \Rightarrow \Gamma_1 \quad \Delta; \Gamma_1; \Theta \vdash^+ \tau_1^{\text{SI}} \leadsto \tau_a^{\text{SI}} \vdash \Gamma_1'$ $\forall r \in \text{free-regions}(\tau_a^{\text{SI}}). \Gamma_1' \vdash r \text{ rnr} \quad \Sigma; \Delta; \Gamma; \text{gc-loans}_{\Theta}(\Gamma_1', x : \tau_a^{\text{SI}}); \Theta \vdash \boxed{e_2} : \tau_2^{\text{SI}} \Rightarrow \Gamma_2, x : \tau^{\text{SD}}$ $\Sigma; \Delta; \Gamma; \Theta \vdash \boxed{\text{let } x : \tau_a^{\text{SI}} = e_1; e_2} : \tau_2^{\text{SI}} \Rightarrow \Gamma_2$	$\epsilon \quad C \vdash \text{drop} : t \rightarrow \epsilon \quad C \vdash \text{select} : t t \text{ i32} \rightarrow t$ $\frac{tf = t_1^n \rightarrow t_2^m \quad C, \text{label}(t_1^n) \vdash e^* : tf}{C \vdash \text{loop } tf \text{ end} : tf}$ $\frac{\vdash e_1^* : tf \quad C, \text{label}(t_2^m) \vdash e_2^* : tf}{e_2^* \text{ end} : t_1^n \text{ i32} \rightarrow t_2^m}$ $\frac{t^*}{32 \rightarrow t^*} \quad \frac{(C_{\text{label}(i)} = t^*)^+}{C \vdash \text{br_table } i^+ : t_1^* t^* \text{ i32} \rightarrow t_2^*}$ $\frac{= tf}{: tf} \quad \frac{tf = t_1^* \rightarrow t_2^* \quad C_{\text{table}} = n}{C \vdash \text{call_indirect } tf : t_1^* \text{ i32} \rightarrow t_2^*}$

Rust (Oxide) [Weiss et. al. 2021]

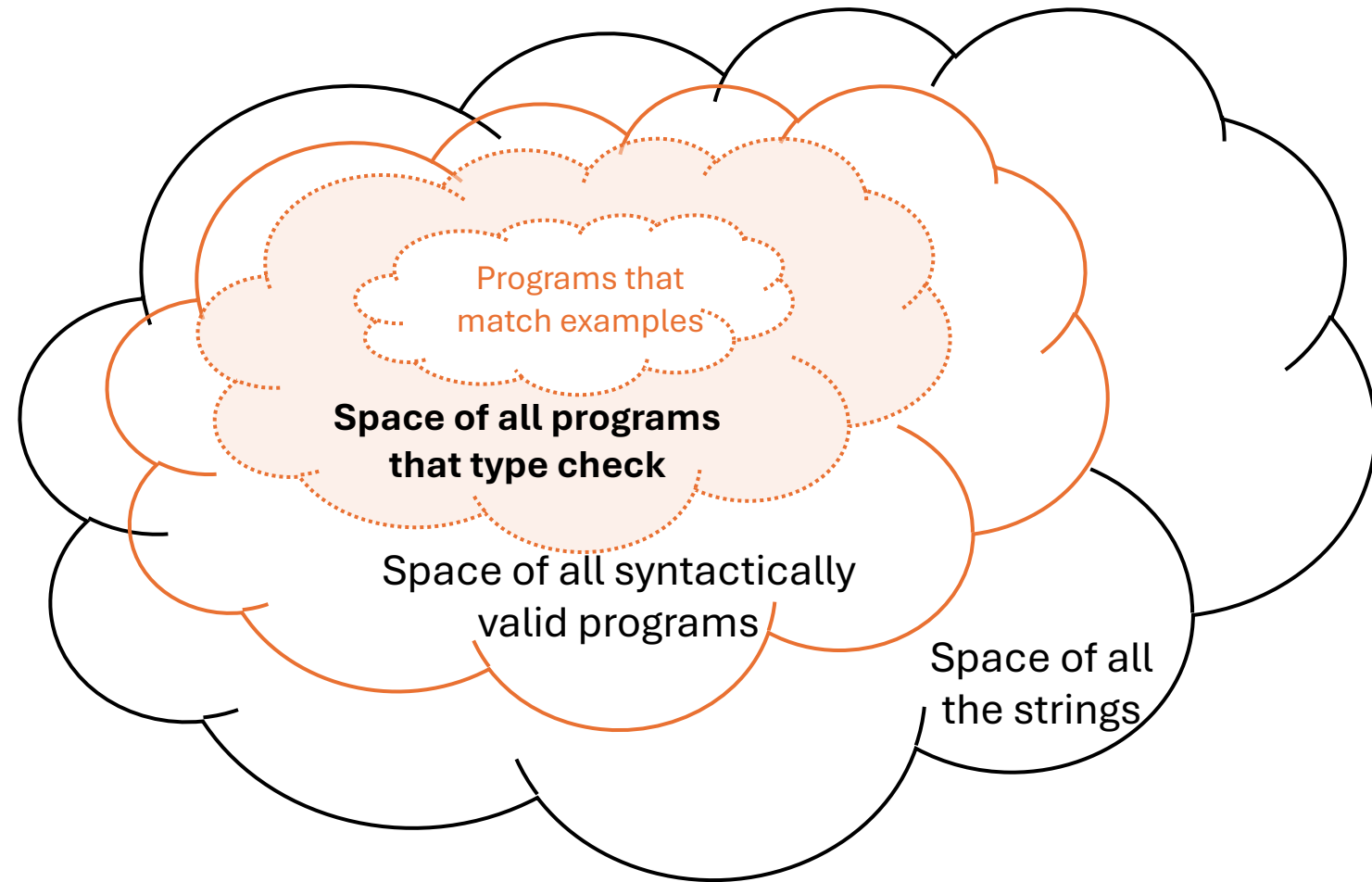
Typing Checking

```
Expr ::= Var  
      | lambda Var: Expr  
      | filter(Expr, Expr)  
      | map(Expr, Expr)  
      | reduce(Expr, Expr, Expr)  
      | Expr if Expr else Expr  
      | Expr BinaryOp Expr  
      | true | false  
      | 0 | 1 | 2 | ...
```



Typing Checking

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```



Top-Down Enumerative Search

dropmin(list) = <Expr>

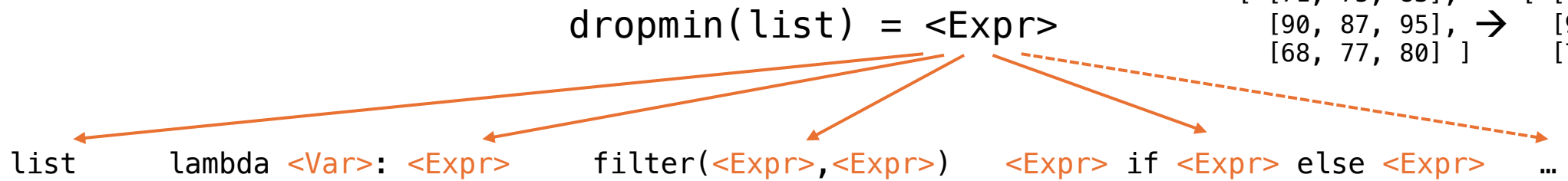
```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```

Top-Down Enumerative Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

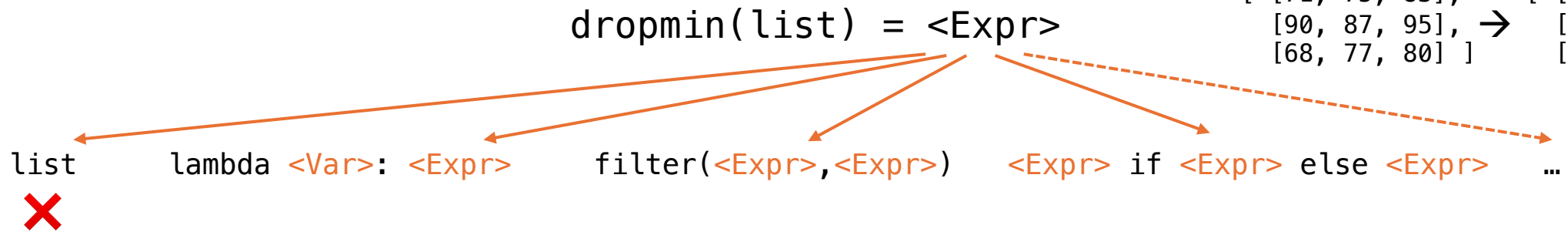
```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```



Top-Down Enumerative Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

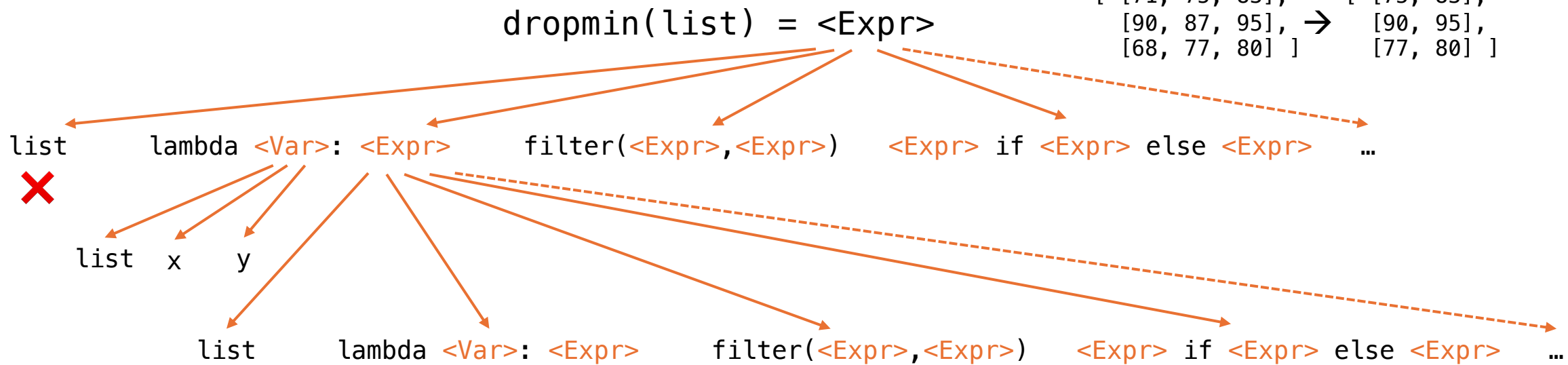
```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```



Top-Down Enumerative Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

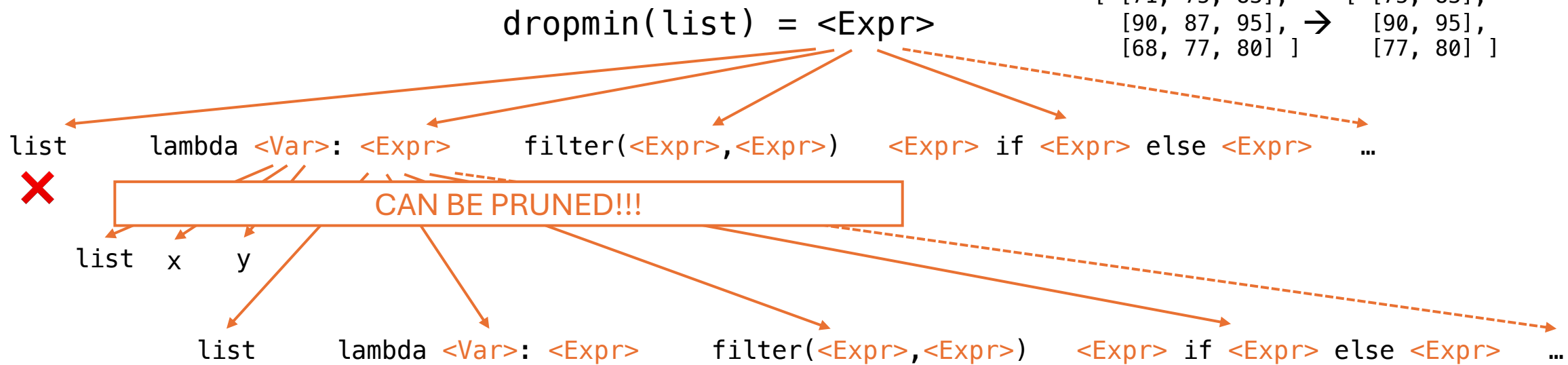
```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```



Top-Down Enumerative Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```



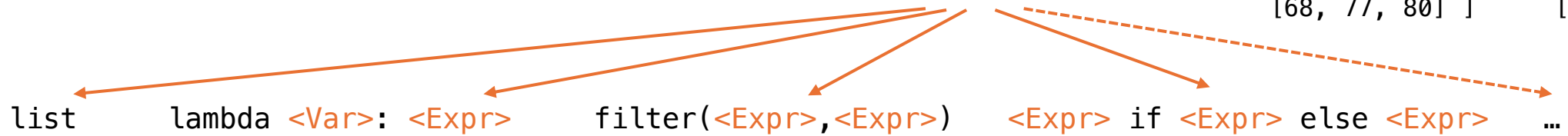
Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>

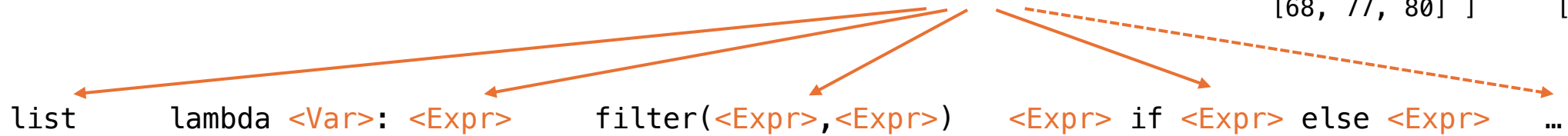
[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]



Type-Guided Top-Down Search

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>



```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```

Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]

list lambda <Var>: <Expr> filter(<Expr>, <Expr>) <Expr> if <Expr> else <Expr> ...

List[List[Int]]

$(list \mapsto \text{List[Int]}) \in \mathcal{C}$

$\mathcal{C} \vdash \text{list} : \text{List[Int]}$

Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]

list lambda <Var>: <Expr> filter(<Expr>, <Expr>) <Expr> if <Expr> else <Expr> ...

List[List[Int]] ✓

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]] ✗

Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]

list

✗

lambda <Var>: <Expr>

filter(<Expr>, <Expr>)

<Expr> if <Expr> else <Expr> ...

Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>

```
[ [71, 75, 83], [75, 83],
  [90, 87, 95], → [90, 95],
  [68, 77, 80] ] [77, 80] ]
```

list
 ✗
 lambda <Var>: <Expr>
List[List[Int]]
 filter(<Expr>, <Expr>) <Expr> if <Expr> else <Expr> ...

$$[\text{Lambda}] \frac{C \cup \{x \mapsto \tau_1\} \vdash e : \tau_2}{C \vdash \lambda x. e : \tau_1 \rightarrow \tau_2}$$

Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[List[Int]] -> List[List[Int]]

dropmin(list) = <Expr>

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]

list

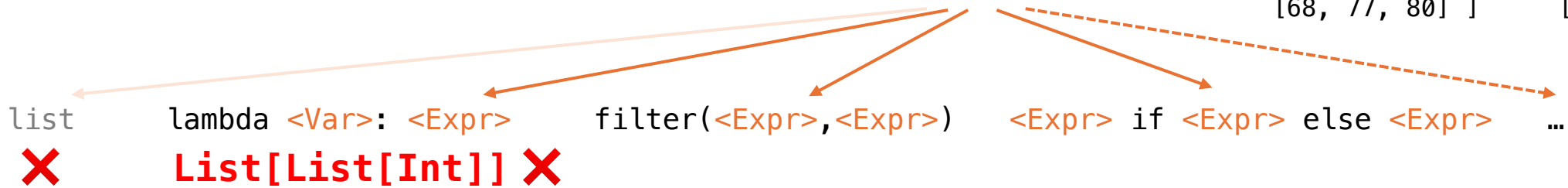
✗

lambda <Var>: <Expr>

List[List[Int]] ✗

filter(<Expr>, <Expr>)

<Expr> if <Expr> else <Expr> ...



Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[Int] -> List[Int]

dropmin(list) = <Expr>

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]

list lambda <Var>: <Expr> filter(<Expr>, <Expr>) <Expr> if <Expr> else <Expr> ...

✗ ✗

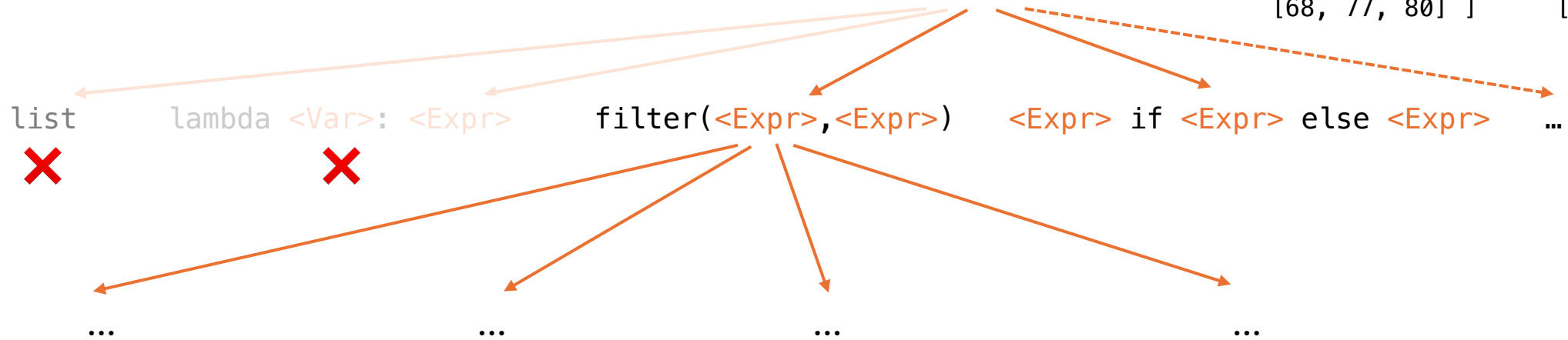
Type-Guided Top-Down Search

```
Expr ::= Var
      | lambda Var: Expr
      | filter(Expr, Expr)
      | map(Expr, Expr)
      | reduce(Expr, Expr, Expr)
      | Expr if Expr else Expr
      | Expr BinaryOp Expr
      | true | false
      | 0 | 1 | 2 | ...
```

dropmin : List[Int] -> List[Int]

dropmin(list) = <Expr>

[[71, 75, 83], [75, 83],
[90, 87, 95], → [90, 95],
[68, 77, 80]] [77, 80]]



Polymorphism

Base types

`Int`

Parametric types (Polymorphism)

`∀T, List[T]`

Parametric type instance (Concrete; Monomorphic)

`List[Int]`

Polymorphism

Base types

`Int`

Parametric types (Polymorphism)

`∀T, List[T]`

Parametric type instance (Concrete; Monomorphic)

`List[Int]`

Types as sets

Base types

`Int` = {..., -2, -1, 0, 1, 2, ...}

Parametric types (Polymorphism)

$\forall T, \text{List}[T]$ = {..., [], ["a", "b"], [3], [false, false, true], ...}

Parametric type instance (Concrete; Monomorphic)

`List[Int]` = {..., [], [1], [2, 3], [1, 2, 3], ...}

Types as sets that can be refined!

Base types

`Int` = {..., -2, -1, 0, 1, 2, ...}

Parametric types (Polymorphism)

$\forall T, \text{List}[T]$ = {..., [], ["a", "b"], [3], [false, false, true], ...}

Parametric type instance (Concrete; Monomorphic)

`List[Int]` = {..., [], [1], [2, 3], [1, 2, 3], ...}

Refinement types

$\{v : \text{Int} \mid v > 0\}$

Types as sets that can be refined!

Base types

`Int` = {..., -2, -1, 0, 1, 2, ...}

Parametric types (Polymorphism)

$\forall T, \text{List}[T]$ = {..., [], ["a", "b"], [3], [false, false, true], ...}

Parametric type instance (Concrete; Monomorphic)

`List[Int]` = {..., [], [1], [2, 3], [1, 2, 3], ...}

Refinement types

$\{v : \text{Int} \mid v > 0\}$ = {1, 2, 3, ...}

Types as sets that can be refined!

Base types

`Int` = {..., -2, -1, 0, 1, 2, ...}

Parametric types (Polymorphism)

$\forall T, \text{List}[T] = \{\dots, [], ["a", "b"], [3], [false, false, true], \dots\}$

Parametric type instance (Concrete; Monomorphic)

`List[Int]` = {..., [], [1], [2, 3], [1, 2, 3], ...}

Refinement types

$\{v : \text{Int} \mid v > 0\} = \{1, 2, 3, \dots\}$

$\forall T, \{ls : \text{List}[T] \mid \text{len}(ls) > 0\} = \{[1], [2, 3], [1, 2, 3], \dots\}$

Refinement Types for Synthesis

“absolute value”

Typing Specification (Original)

`abs : Int -> Int`

Typing Specification (with Refinement)

`abs : Int -> {v : Int | v >= 0}`

Refinement Types for Synthesis

“duplicate every element in a list”

Typing Specification (Original)

```
stutter : List[T] -> List[T]
```

Typing Specification (with Refinement)

```
stutter : in: List[T] -> {v: List[T] | len(v) = 2 * len(x)}
```

Refinement Types for Synthesis

“sort the elements in an array”

Typing Specification (Original)

```
sort : List[T] -> List[T]
```

Typing Specification (with Refinement)

```
sort : List[T] -> {v: List[T] | is_sorted(v)}
```

Refinement Types for Synthesis

“sort the elements in an array”

Typing Specification (Original)

```
sort : List[T] -> List[T]
```

Typing Specification (with Refinement)

```
sort : List[T] -> {v: List[T] | is_sorted(v)}  
is_sorted(v) =  $\forall i, j, \text{ s.t. }, 0 \leq i < j < \text{len}(v), v[i] \leq v[j]$ 
```

Refinement Types for Synthesis

“insert an element into an array”

Typing Specification (Original)

```
insert : T -> List[T] -> List[T]
```

Typing Specification (with Refinement)

```
insert : (e: T) -> (x: List[T]) -> {y : List[T] | elems(y) = elems(x) U {e}}
```

How to Enforce Constraints in Refinement Types

- similar strategies in **top-down** enumerative synthesis
- with type checkers capable of **checking** and **strengthening** refinement types
- during the type checking, we may rely on **human defined predicates** such as “len” and “elems”

How to Enforce Constraints in Refinement Types

Leveraging Rust Types for Program Synthesis

JONÁŠ FIALA, Department of Computer Science, ETH Zurich, Switzerland

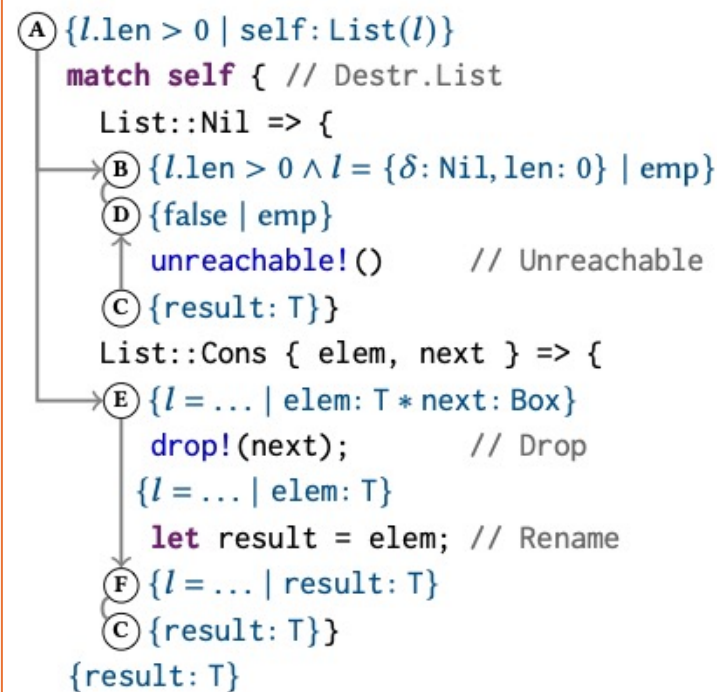
SHACHAR ITZHAKY, Technion, Israel

PETER MÜLLER, Department of Computer Science, ETH Zurich, Switzerland

NADIA POLIKARPOVA, University of California, San Diego, USA

ILYA SERGEY, National University of Singapore, Singapore

```
#[ensures((^self).len() ==  
          (*self).len() + (^result).len())]  
fn last_mut(&mut self) -> &mut List<T> {  
  todo!()  
}
```



Summary

- Functional Language with Higher-Order Functions
- Type System, Typing Rules, and Type Checking
- Top-down Enumerative Synthesis Guided by Types
- (A little bit of) Refinement Types for Synthesis

Week 2

- Assignment 1
 - Released: <https://github.com/machine-programming/assignment-1>
 - Due Tuesday of the Forth Week (Sept 22)
 - AutoGrader not up yet; will be later today
- Any questions?